

LAST 4-WEEK REPORT

Weather Station:

AIoT Multi-Modal Sensing Overview



Under the Supervision of:

Dr. T V PRABHAKAR
IISC-DESE, BANGLORE

Submitted by

SHAIK HAIDAR
ELECTRONICS AND COMMUNICATION DEPARTMENT
UCEK-JNTUK (Autonomous), KAKINADA
Application No:ENG5618

1 Introduction

Environmental monitoring has become increasingly important for understanding atmospheric behavior, supporting climate studies, improving agricultural practices, enabling disaster preparedness, and facilitating the development of smart cities. Continuous observation of environmental parameters such as temperature, humidity, atmospheric pressure, particulate matter concentration, rainfall, wind speed, wind direction, and ambient light provides valuable insights into changing weather conditions and environmental quality. Conventional weather monitoring systems primarily focus on data collection and visualization; however, the growing availability of Artificial Intelligence (AI), Internet of Things (IoT), and edge computing technologies has created opportunities to develop more intelligent systems capable of not only acquiring environmental data but also interpreting, analyzing, and predicting weather conditions in real time.

This project presents an AIoT-based Multi-Modal Weather Monitoring and Intelligent Weather Assistant System that combines an embedded weather station with a conversational AI chatbot. The weather monitoring subsystem is built around a Raspberry Pi 4 and integrates multiple environmental sensors to continuously measure key atmospheric parameters, including temperature, humidity, pressure, particulate matter, rainfall, wind speed, wind direction, and ambient light intensity. Sensor observations are processed locally at the edge and transmitted using the lightweight MQTT communication protocol through a Mosquitto Broker. The collected data are stored for long-term analysis and visualized through an interactive dashboard, providing users with real-time access to environmental information while ensuring reliable and efficient data management.

To enhance user interaction and extend the capabilities of the weather station, the project incorporates WeatherGPT (WGPT), an AI-powered weather intelligence assistant that enables users to access weather information through natural language conversations. The chatbot combines Natural Language Processing (NLP), machine learning, and Large Language Models (LLMs) to interpret user queries, retrieve historical weather observations, perform statistical analysis, and generate short-term weather forecasts. Historical weather data are obtained from locally stored datasets, while future weather conditions are predicted using a Long Short-Term Memory (LSTM) neural network trained on historical weather records. Retrieved data and prediction results are converted into conversational responses using a locally hosted Ollama-based LLM, with a rule-based response mechanism serving as a fallback to ensure uninterrupted offline operation.

The overall system follows a modular architecture that separates sensing, data acquisition, communication, storage, artificial intelligence, and user interaction into independent functional layers. This design improves scalability, simplifies maintenance, and allows additional sensors, forecasting models, and AI services to be incorporated with minimal modifications. Furthermore, the system supports multiple user interfaces, including a command-line chatbot, a web-based dashboard with REST APIs, and a Model Context Protocol (MCP) server, enabling seamless integration with external AI applications and intelligent assistants.

By integrating edge computing, IoT communication, machine learning, and conversational AI into a unified platform, the proposed system demonstrates an efficient and cost-effective approach for intelligent environmental monitoring. Beyond conventional weather stations, it provides users with an interactive environment capable of real-time monitoring, historical data exploration, and predictive weather analysis, making it well suited for research, educational, and smart environmental monitoring applications.

The remainder of this report describes the system architecture, hardware and software implementation, sensor integration, communication framework, chatbot design, forecasting methodology, and experimental evaluation of the proposed AIoT-based intelligent weather monitoring system.

2 Abstract

This work presents WeatherGPT, an AI-assisted weather monitoring and forecasting framework that integrates Artificial Intelligence (AI), Internet of Things (IoT), edge computing, and conversational intelligence into a unified environmental monitoring platform. The system employs a Raspberry Pi 4 integrated with multiple environmental sensors to continuously monitor temperature, humidity, atmospheric pressure, particulate matter, rainfall, wind speed, wind direction, and ambient light intensity. Sensor observations are processed at the edge, transmitted using the MQTT communication protocol, archived for historical analysis, and visualized through an interactive dashboard. A custom Natural Language Processing (NLP) pipeline enables users to retrieve historical weather information, perform statistical analysis, compare weather conditions across multiple dates, and interact with the system using natural language.

Short-term weather forecasting is performed using a stacked Long Short-Term Memory (LSTM) neural network implemented in PyTorch and trained on historical weather observations to generate multivariate forecasts for up to 30 days. The framework further integrates locally hosted Large Language Models (LLMs), REST APIs, command-line and web-based interfaces, and Model Context Protocol (MCP) support to provide reliable offline operation and seamless interoperability with external AI applications. Building upon this foundation, the research is being extended towards the development of an advanced WeatherBot based on Graph Neural Networks (GNNs) to model spatial and temporal relationships among multiple sensing nodes, enabling scalable multi-node weather prediction and intelligent environmental analysis for next-generation AIoT weather monitoring systems.

3 key words

WeatherGPT, Artificial Intelligence (AI), Natural Language Processing (NLP), Conversational AI, Weather Forecasting, Historical Weather Data Retrieval, Time Series Forecasting, Long Short-Term Memory (LSTM), PyTorch, Ollama, Decision Engine, Offline AI Assistant, Information Retrieval, SQLite Database, REST API, Model Context Protocol (MCP), Weather Analytics, Bangalore Weather Dataset.

4 summary of first 4 weeks of internship

During the first four weeks of the Summer Research Fellowship, the primary focus was on understanding the architecture and operation of an intelligent weather monitoring system. This involved studying the Raspberry Pi 4B platform, environmental sensors, communication protocols, network time synchronization, and data quality metrics such as AoI and VAoI. A comprehensive review of relevant literature was also carried out to understand the system architecture, data flow, and edge-based processing, providing a strong foundation for the project.

Based on this understanding, the scope of the project was extended by exploring potential improvements to the existing weather station. This included studying additional sensing and synchronization techniques, evaluating power-monitoring components, and investigating energy optimization approaches such as Dynamic Voltage and Frequency Scaling (DVFS) for Raspberry Pi systems. These studies established the technical groundwork for the implementation and evaluation phases planned for the remainder of the internship.

5 About the weather station

The proposed AIoT Multi-Modal Weather Station is designed as a comprehensive environmental monitoring platform capable of continuously sensing, processing, storing, and visualizing multiple atmospheric parameters in real time. Unlike conventional weather stations that primarily focus on a limited set of meteorological variables, this system integrates multiple sensing modalities to provide a holistic understanding of environmental conditions. The architecture combines heterogeneous sensors, edge computing, lightweight communication protocols, local storage, and visualization techniques to create a reliable and scalable monitoring solution.

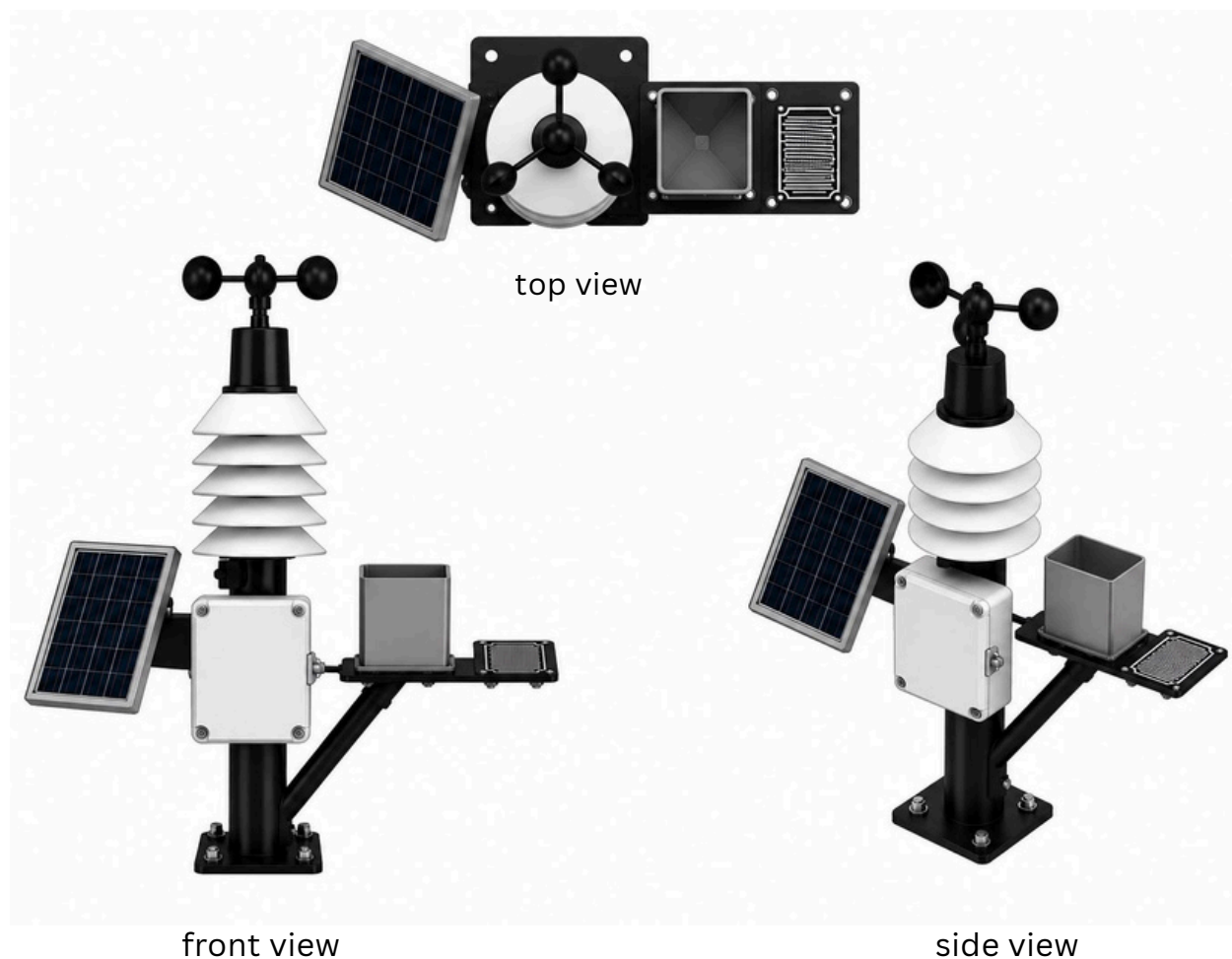


Fig : model of the weather station

5.1 Project Overview

Environmental monitoring has become increasingly important due to rapid climate change, urbanization, and growing concerns regarding air quality and weather-related hazards. Accurate measurement of atmospheric parameters enables researchers and engineers to analyze weather patterns, monitor pollution levels, and develop intelligent environmental monitoring systems.

The proposed AIoT Multi-Modal Weather Station addresses these requirements by integrating multiple environmental sensors with a Raspberry Pi 4-based processing unit. The collected sensor readings are processed locally and transmitted using the MQTT communication protocol before being archived in CSV format for visualization and future analysis. The system is capable of continuously monitoring multiple weather parameters while maintaining a lightweight and energy-efficient architecture suitable for research and field deployments.

The modular implementation ensures flexibility in hardware expansion and software enhancements while providing a robust platform for continuous environmental monitoring.

5.2 Objectives

The primary objectives of the proposed weather monitoring station are summarized below:

- To continuously monitor multiple environmental parameters in real time.
- To integrate heterogeneous weather sensors into a unified monitoring platform.
- To perform reliable local acquisition and preprocessing of sensor observations.
- To establish lightweight communication using the MQTT publish-subscribe protocol.
- To maintain local archival of weather observations using CSV-based storage.
- To provide intuitive visualization of environmental data through a dashboard interface.
- To develop a modular and scalable AIoT architecture suitable for future research applications.

5.3 Implemented System Architecture

The implemented weather station follows a layered architecture designed to simplify data acquisition, communication, storage, and visualization. Each subsystem performs a dedicated function while collectively contributing to reliable environmental monitoring

The major components of the implemented architecture include:

- Sensor Layer
- Raspberry Pi 4 (Data Acquisition and Local Processing)
- MQTT Communication Layer
- Mosquitto MQTT Broker
- CSV Data Logging
- Dashboard Visualization
- End Users

The interaction among these components enables continuous monitoring of atmospheric conditions while maintaining efficient data handling and minimal communication overhead.

5.3.1 Sensor Layer

The Sensor Layer forms the foundation of the AIoT weather station by continuously acquiring environmental observations from the surrounding atmosphere. Multiple sensors are integrated to capture different meteorological and air-quality parameters, thereby providing a comprehensive representation of weather conditions.

The implemented sensing devices include:

- BME280 for measuring temperature, relative humidity, and atmospheric pressure.
- SPS30 for measuring particulate matter concentrations including PM1.0, PM2.5, PM4.0, and PM10.
- YL-83 for detecting the presence of rainfall.
- Tipping Bucket Rain Gauge for measuring accumulated rainfall.
- Anemometer for monitoring wind speed.
- Wind Vane for measuring wind direction.

Each sensor communicates with the Raspberry Pi through suitable interfaces such as I²C, UART, or GPIO depending on its communication protocol. The combination of multiple sensing modalities enables comprehensive environmental monitoring under varying atmospheric conditions.

5.3.2 Raspberry Pi 4 (Data Acquisition and Local Processing)

The Raspberry Pi 4 serves as the central processing unit of the proposed weather station. It acts as the interface between the sensing layer and the communication infrastructure by periodically acquiring measurements from all connected sensors.

Upon receiving sensor observations, the Raspberry Pi performs several preprocessing operations, including sensor polling, data formatting, timestamp generation, and validation of acquired measurements. These preprocessing steps ensure that only structured and meaningful environmental information is forwarded to subsequent stages of the system.

Local processing minimizes computational overhead on downstream applications while improving the overall reliability and efficiency of the weather monitoring platform.

Table: Integrated Sensors and Their Specifications

s.no	Sensor Name	Measured Parameter(s)	Communication Interface	Purpose
1	BME280	Temperature, Relative Humidity, Atmospheric Pressure	I ² C	Environmental monitoring and weather analysis
2	SPS30	PM1.0, PM2.5, PM4.0, PM10	UART	Air quality and particulate matter monitoring
3	YL-83 Rain Sensor	Rain Presence Detection	GPIO	Detection of rainfall events
4	Tipping Bucket Rain Gauge	Rainfall Amount	GPIO	Quantitative rainfall measurement
5	Anemometer	Wind Speed	GPIO	Wind speed monitoring
6	Wind Vane	Wind Direction	GPIO	Wind direction estimation

5.3.3 MQTT Communication Layer

Following local processing, the acquired environmental observations are transmitted using the Message Queuing Telemetry Transport (MQTT) protocol.

MQTT is a lightweight publish-subscribe communication protocol specifically designed for Internet of Things applications requiring minimal bandwidth consumption and reliable communication.

Within the implemented architecture, the Raspberry Pi functions as the MQTT Publisher by periodically publishing processed weather observations to predefined communication topics. The publish-subscribe model eliminates direct coupling between data producers and consumers, thereby improving scalability and communication efficiency.

5.3.4 Mosquitto MQTT Broker

The Mosquitto Broker functions as the communication hub of the proposed system. It receives MQTT messages published by the Raspberry Pi and manages their routing toward the storage subsystem.

The broker is responsible for:

Receiving published environmental observations.

Managing MQTT communication topics.

Forwarding weather data to subscribed applications.

Ensuring reliable message delivery throughout the communication process.

The use of Mosquitto simplifies communication management while enabling efficient data exchange between hardware and software components.

5.3.5 CSV Data Logging

To preserve environmental observations for future reference, all incoming weather data are archived locally as CSV files.

CSV-based storage provides a lightweight and platform-independent mechanism for maintaining historical environmental records. This approach enables efficient retrieval of previously collected observations while simplifying data analysis and visualization.

The local storage mechanism also serves as a backup repository, ensuring that valuable sensor data remain available even during temporary communication interruptions.

5.3.6 Dashboard Visualization

The visualization dashboard provides a user-friendly graphical interface for observing environmental conditions in real time.

The dashboard continuously retrieves stored weather observations from the CSV files and presents them through graphical charts and numerical indicators. The visualization interface enables users to monitor current weather conditions while simultaneously analyzing historical environmental trends.

The dashboard visualizes the following parameters:

- Temperature
- Relative Humidity
- Atmospheric Pressure
- PM1.0
- PM2.5
- PM4.0
- PM10
- Rain Detection
- Rainfall Amount
- Wind Speed
- Wind Direction
- Light Intensity

The visualization platform significantly improves the interpretability of collected weather information and supports efficient environmental monitoring.

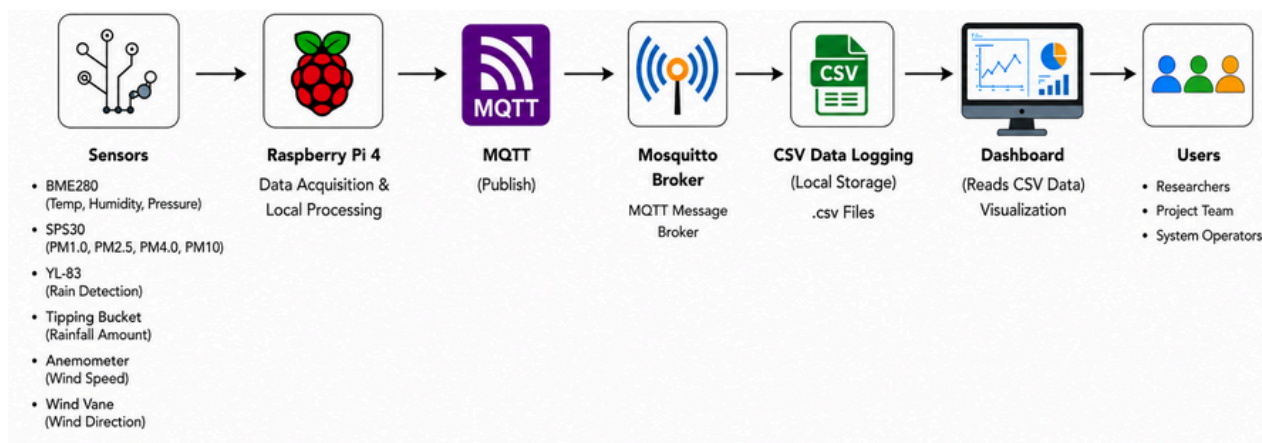
5.3.7 End Users

The final component of the implemented architecture consists of the end users who interact with the weather monitoring station through the visualization dashboard.

The dashboard enables these users to continuously monitor environmental conditions, analyze historical observations, and evaluate overall system performance through an intuitive graphical interface.

5.4 Overall Data Flow

The operational workflow of the implemented AIoT Multi-Modal Weather Station is illustrated in Figure given below. The data flow begins with the environmental sensors, which continuously acquire atmospheric observations. These measurements are transferred to the Raspberry Pi 4, where local data acquisition and preprocessing are performed. The processed sensor readings are then published using the MQTT protocol and forwarded to the Mosquitto MQTT Broker. The processed sensor readings are then published using the MQTT protocol and forwarded to the Mosquitto MQTT Broker. The processed sensor readings are then published using the MQTT protocol and forwarded to the Mosquitto MQTT Broker.



Rather than storing the observations in a remote database, the proposed implementation maintains local CSV-based storage, where all environmental measurements are archived for future retrieval. The visualization dashboard periodically reads the stored CSV files and presents the information through interactive graphical interfaces. Finally, researchers, developers, and system operators access the dashboard to observe both real-time and historical environmental conditions.

The complete workflow of the proposed weather monitoring station can be summarized as follows:

- Environmental sensors continuously monitor atmospheric conditions.
- The Raspberry Pi 4 acquires and locally processes the sensor observations.
- Processed weather data are published using the MQTT protocol.
- The Mosquitto MQTT Broker receives and manages the published messages.
- Sensor observations are archived locally in CSV files.
- The dashboard retrieves weather information from the CSV files.
- End users visualize and analyze the environmental observations through the dashboard.

6 work done

6.1 Analysis of Weather Station State Architecture

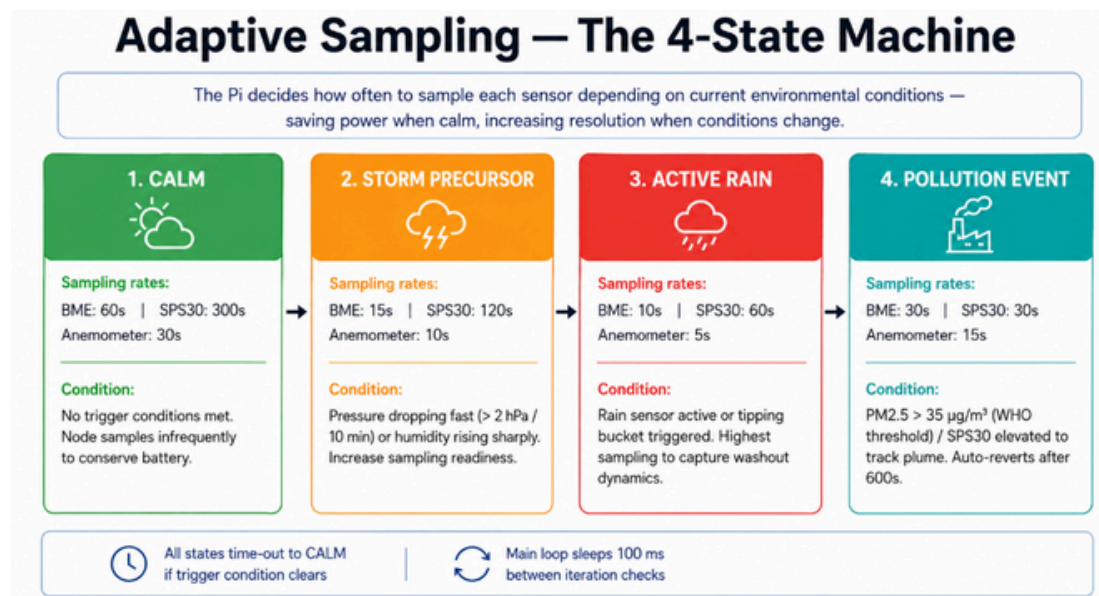
The initial phase of the second half of the internship focused on understanding and evaluating the operational state architecture of the weather station. Before introducing artificial intelligence and predictive capabilities into the system, it was necessary to understand how the existing weather station monitored environmental conditions and generated alerts based on sensor observations.

A detailed analysis was conducted on the state transition mechanism implemented within the weather station. The objective of this study was to determine whether the existing four-state alerting framework was sufficient for reliable operation or whether additional intermediate states would improve the accuracy and responsiveness of the monitoring system.

The functionality of each operational state was examined by studying the corresponding transition conditions, threshold values, and alert generation logic. Particular attention was given to understanding how changes in meteorological parameters triggered transitions between different operational states and how these transitions influenced the decision-making process of the monitoring system.

Different state configurations were evaluated conceptually by considering possible scenarios such as gradual environmental changes, sudden fluctuations in weather conditions, and sensor noise. The analysis also included studying the impact of introducing additional alerting states on system complexity, computational overhead, maintainability, and interpretability.

Based on the observations obtained during this analysis, it was concluded that the existing four-state architecture was capable of representing the required operational conditions effectively for the intended application. Introducing additional states would increase the complexity of the system without providing a proportionate improvement in monitoring performance. Therefore, the existing state model was retained as the foundation for the subsequent development of the AI-assisted weather prediction framework.



The knowledge acquired during this phase provided a comprehensive understanding of the weather station's operational workflow and served as a prerequisite for integrating machine learning and conversational intelligence into the existing architecture.

6.2 Literature Review and Investigation of Optimization Techniques

Following the analysis of the weather station architecture, an extensive literature review was conducted to investigate recent advancements in weather prediction systems, optimization methodologies, and artificial intelligence techniques applicable to meteorological forecasting.

Research papers from journals, conference proceedings, and technical publications were reviewed to understand the evolution of weather forecasting methods, ranging from conventional statistical approaches to modern machine learning and deep learning techniques. The review primarily focused on methodologies used for time-series prediction, sequence modelling, feature engineering, optimization strategies, and intelligent decision-support systems.

Particular emphasis was placed on identifying optimization techniques capable of improving prediction accuracy while maintaining computational efficiency. Various studies were examined to understand the influence of data preprocessing, feature normalization, hyperparameter tuning, learning rate optimization, loss function selection, and model regularization on forecasting performance. Different optimization algorithms and training strategies reported in the literature were also compared to understand their advantages and limitations in weather prediction applications.

In addition to optimization methods, the literature survey included a comparative study of several predictive algorithms commonly used in meteorological forecasting, including Linear Regression, Decision Trees, Random Forests, Support Vector Machines (SVM), Artificial Neural Networks (ANN), Recurrent Neural Networks (RNN), Gated Recurrent Units (GRU), and Long Short-Term Memory (LSTM) networks. Each algorithm was evaluated based on its ability to model nonlinear relationships, capture temporal dependencies, process sequential weather observations, and generate accurate long-term forecasts.

The review indicated that recurrent deep learning architectures consistently outperformed conventional machine learning models when applied to multivariate weather time-series datasets. Among these, the Long Short-Term Memory (LSTM) network demonstrated superior capability in learning long-range temporal dependencies while effectively addressing the vanishing gradient problem encountered in conventional recurrent neural networks. later due to its simplicity in designing made it obvious to choose the lstm algorithm to develop a weather predicting chatbot

Overall, the literature review established the theoretical foundation for the project by identifying state-of-the-art forecasting methodologies, evaluating existing optimization techniques, and providing a systematic basis for the architectural and algorithmic decisions made during the development of the intelligent weather prediction system

6.3 Development of the WeatherGPT Framework

Following the selection of the LSTM model and the overall system architecture, the implementation focused on developing a complete AI-assisted weather prediction framework, referred to as WeatherGPT. The objective was to integrate natural language understanding, machine learning-based weather forecasting, and conversational artificial intelligence into a unified system capable of providing accurate and user-friendly weather information.

6.3.1 Development of the Core Processing Engine

The first stage of development concentrated on designing the core processing engine, which serves as the intelligence layer of the WeatherGPT framework. This layer was responsible for understanding user queries, retrieving relevant weather information, generating forecasts, and producing natural language responses.

A custom Natural Language Processing (NLP) pipeline was developed to process user queries efficiently. The pipeline consisted of a tokenizer, an intent classification module, and an entity extraction mechanism. The tokenizer segmented user input into meaningful tokens, while the intent classifier identified the purpose of the query, such as requesting weather information for a specific date, obtaining statistical summaries, or generating future weather forecasts. The entity extraction module identified temporal information, including dates and time periods, enabling the system to retrieve relevant weather records accurately.

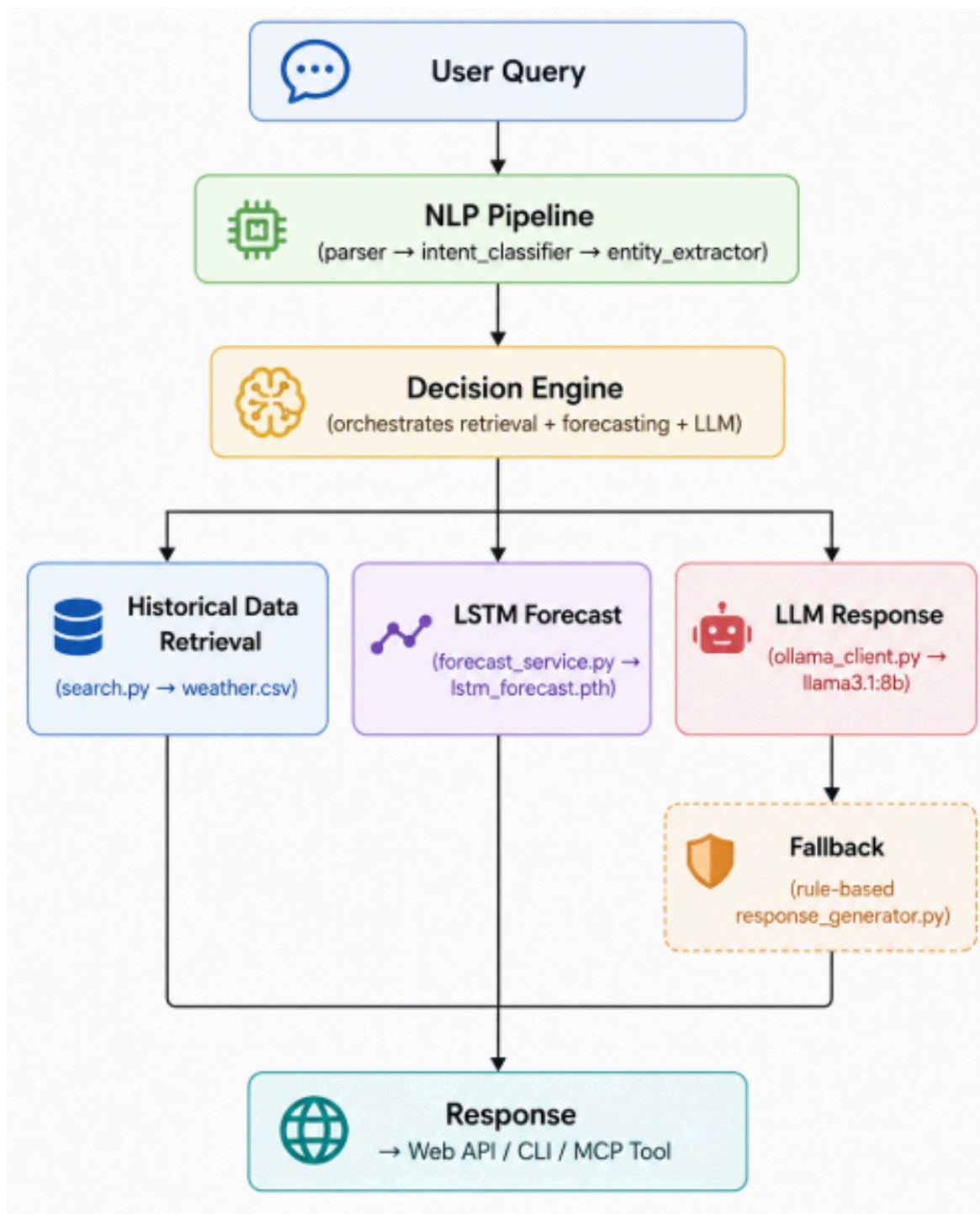


Fig : Architecture of the LSTM weather chatbot

To support efficient data retrieval, a dynamic data loading framework was implemented using Python. Historical weather datasets were automatically imported into Pandas DataFrames, and the system was designed to detect and process date-related attributes without requiring manual configuration.

Dedicated retrieval modules were developed to filter, search, and extract weather records based on the temporal entities identified by the NLP pipeline. This modular design significantly improved the flexibility and scalability of the retrieval process.

6.4 Integration of Large Language Models

To provide conversational responses, the prediction engine was integrated with a Large Language Model (LLM). Initially, Ollama 3.1 (8B) was employed for local inference during development. Subsequently, Groq-based inference was incorporated to enable faster response generation and improved conversational performance.

To enhance the robustness of the application, a multi-level response generation mechanism was implemented. If the primary language model was unavailable, the system automatically switched to an Ollama-based inference engine.

In situations where neither language model was accessible, a rule-based response generation module ensured uninterrupted system operation. This fallback architecture significantly improved system reliability and fault tolerance.

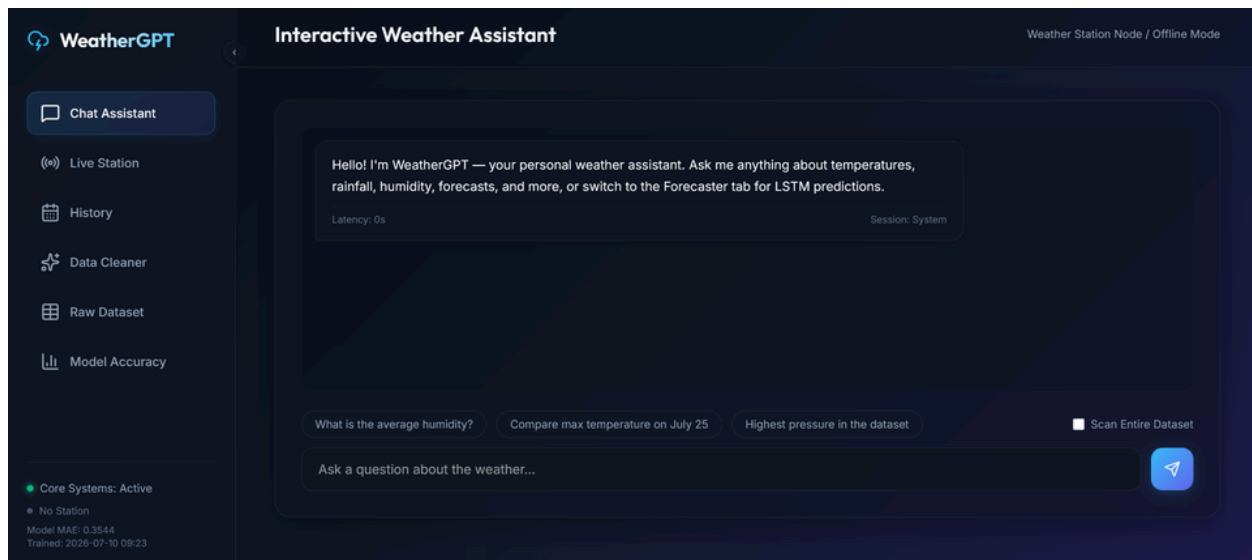
6.5 Development of User Interfaces and Application Framework

After validating the functionality of the core processing engine, the next phase involved transforming the prediction framework into a complete software application that could be accessed through multiple interfaces.

6.6 Web-Based Dashboard

A web-based dashboard was developed to provide an intuitive interface for interacting with the WeatherGPT system. A lightweight RESTful API was implemented using Python-based web technologies to expose endpoints responsible for chatbot interaction and weather prediction services.

The frontend was developed as a Single-Page Application (SPA) using HTML, CSS, and JavaScript. The dashboard enabled users to interact with the chatbot through a conversational interface while simultaneously displaying system information and weather prediction results. Interactive visualizations were incorporated to present forecasted weather trends graphically, thereby improving the interpretability of prediction outputs.



6.7 Command-Line Interface

In addition to the graphical web interface, a command-line application was developed to facilitate direct interaction with the WeatherGPT framework from the terminal. This interface proved particularly useful during system testing, debugging, and rapid experimentation.

To simplify execution across different development environments, an automated environment detection mechanism was implemented. The application verified whether the correct Python virtual environment was active and, if necessary, automatically relaunched itself within the configured virtual environment. This approach minimized dependency-related execution errors and improved the portability of the application.

6.8 Model Context Protocol (MCP) Integration

To facilitate seamless integration between the AI-powered chatbot and the AIoT-based weather monitoring station, the Model Context Protocol (MCP) was adopted as the communication framework.

MCP provides a standardized protocol that enables AI applications to securely interact with external tools, services, and data sources through a unified interface. By eliminating the need for custom integration logic, MCP simplifies communication between the chatbot and the weather station while improving interoperability and maintainability.

In the proposed system, an MCP-compatible server was developed using the FastMCP framework to expose the weather station's functionalities as callable tools. Through this server, the chatbot can communicate directly with the weather station to access real-time sensor observations, retrieve historical weather records, and invoke weather-related services using standardized MCP requests. The MCP server acts as an intermediary between the chatbot and the weather station, managing request routing, tool execution, and response delivery in a structured and consistent manner.

The adoption of MCP enhances the modularity, scalability, and extensibility of the overall architecture by providing a common communication layer that decouples the chatbot from the underlying weather station implementation. This approach allows additional AI applications, intelligent agents, or external development environments supporting MCP to integrate with the weather station without requiring modifications to the existing system. Consequently, the weather station can be utilized as a reusable AI service, enabling efficient access to environmental data and weather-related functionalities through a standardized protocol while supporting future expansion of the system.

6.9 Cloud Deployment Architecture

The final stage of development focused on deployment and accessibility. The complete application, including the backend server, REST API, prediction engine, chatbot, and frontend dashboard, was configured for cloud deployment.

Deployment configuration files were created to enable seamless hosting on cloud platforms such as Render. This deployment architecture ensured that the WeatherGPT framework could be executed as an online application without requiring extensive manual configuration.

The modular deployment strategy also facilitates future scalability, maintenance, and integration with real-time weather data sources.

6.10 System Integration

Upon completion of all development phases, the individual components—including the NLP pipeline, historical weather database, LSTM prediction model, conversational language model, retrieval engine, web dashboard, command-line interface, and MCP server—were integrated into a unified WeatherGPT framework.

The resulting system provides end-to-end functionality by accepting natural language weather queries, retrieving relevant historical information, generating AI-based weather forecasts, and presenting results through multiple user interfaces. This integrated architecture demonstrates the practical application of deep learning, natural language processing, and modern software engineering principles in the development of an intelligent weather forecasting assistant.

6.12 Training of the chatbot for the weather prediction

The weather prediction component of WeatherGPT was implemented using a Stacked Long Short-Term Memory (LSTM) neural network developed in the PyTorch deep learning framework. The model was designed to capture temporal dependencies present in sequential weather observations and generate accurate multivariate weather forecasts based on historical weather data. During the development and validation of the chatbot, mock weather readings were used to train, validate, and verify the prediction pipeline. These simulated datasets enabled testing of the chatbot's forecasting functionality and ensured that it could correctly process user queries and generate weather predictions before deployment with live weather station data. By learning patterns from historical weather observations, the model is capable of predicting multiple meteorological parameters for a forecasting horizon of up to 30 days. Multiple training iterations were performed to optimize the network parameters and improve the overall predictive performance. Upon completion of training, the optimized model was integrated into the WeatherGPT framework, enabling real-time weather forecasting during chatbot interactions.

6.12.1 Model Architecture

The forecasting model is based on a Stacked LSTM architecture consisting of two LSTM layers, each with 64 hidden units. The stacked configuration enables the network to learn both short-term and long-term temporal relationships within historical weather sequences. A selective skip connection is incorporated to preserve the most recent values of relatively stable weather variables, thereby improving prediction stability. Furthermore, a dropout rate of 0.2 is applied between the LSTM layers to reduce overfitting and enhance the model's generalization capability.

6.12.2 Training Hyperparameters

The model was trained using a predefined set of hyperparameters to achieve efficient convergence and reliable forecasting performance. A lag window of five days was used, allowing the network to learn from the weather observations recorded during the previous five days. The training process was carried out for a maximum of 200 epochs using a batch size of 64. Model parameters were optimized using the Adam optimizer with a learning rate of 0.003, enabling stable and efficient learning throughout the training process.

6.12.3 Data Preparation

Prior to training, the historical weather dataset underwent preprocessing to improve the learning capability of the model. Feature engineering techniques, including cyclical encoding of time-based variables and rolling statistical features, were applied to capture seasonal patterns and recent weather trends. The processed data was then normalized using StandardScaler, with separate scalers maintained for the input features and target variables. Finally, the dataset was divided sequentially into 90% training data and 10% validation data, preserving the chronological order of observations for realistic model evaluation.

6.12.4 Loss Function

To optimize multivariate weather prediction, the model employed a custom Weighted L1 Loss (Weighted Mean Absolute Error) function. Since multiple weather variables are predicted simultaneously, different static weights were assigned to individual output parameters according to their importance. Higher weights were allocated to critical variables such as temperature and humidity, enabling the model to prioritize accurate prediction of the most significant weather metrics.

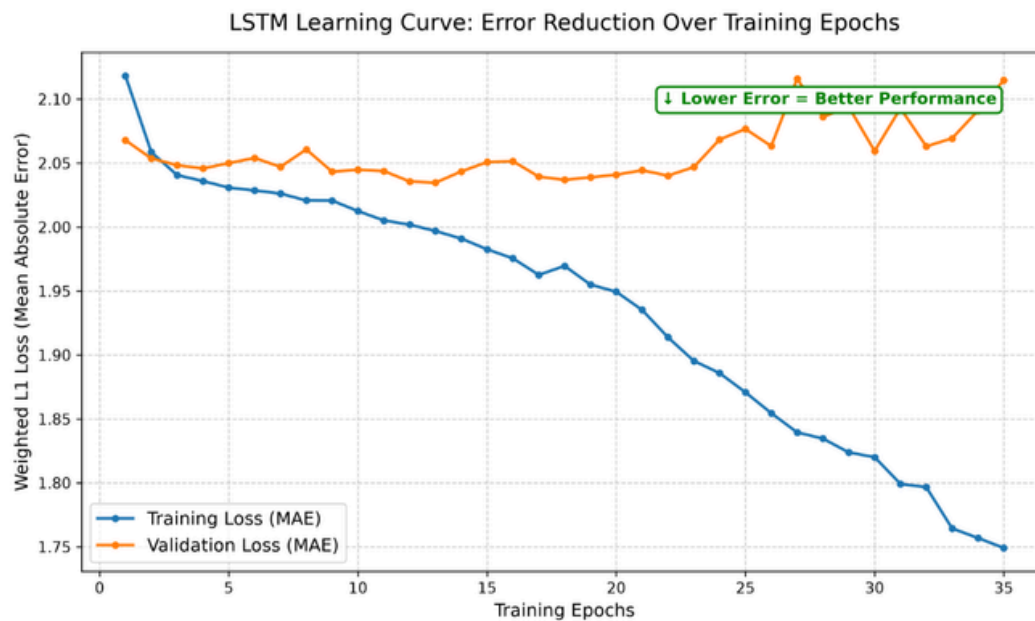
Weather Feature	MAE (Original Units)
temp_max	1.0404 °C
temp_min	0.4678 °C
temp	0.4534 °C
feelslike	0.5955 °C
humidity	2.5185 %
dew	0.3734 °C
precip	6.0472 mm
precip_prob	40.2123 %
windgust	12.0691 km/h
windspeed	20.1067 km/h
winddir	85.1951 degrees
pressure	0.5888 hPa
cloudcover	15.6895 %
visibility	0.9058 km
uvindex	1.9462 rating

Table : MAE readings of mock data used

6.12.5 Training Process

The training process involved repeatedly feeding sequential weather observations into the stacked LSTM network while minimizing the weighted prediction error. During each epoch, model parameters were updated using the Adam optimization algorithm.

To prevent overfitting, an Early Stopping mechanism with a patience of 25 epochs was incorporated. If the validation loss failed to improve for 25 consecutive epochs, the training process terminated automatically, and the model restored the weights corresponding to the best-performing validation epoch.



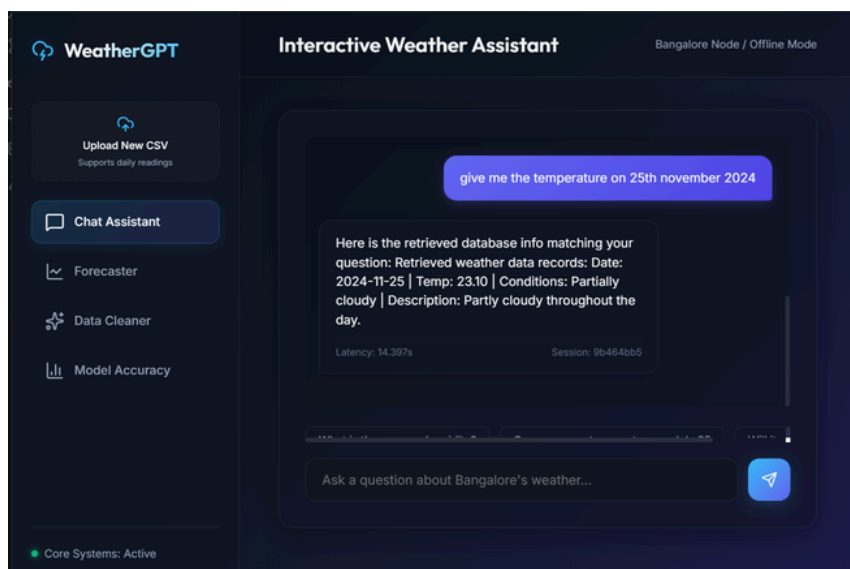
This strategy ensured that the final model achieved optimal prediction performance without unnecessary training.

The system also supports background retraining, allowing the prediction model to be updated with newly available weather observations without interrupting chatbot operation. By default, background retraining is performed for 150 epochs, ensuring that the forecasting model remains adaptive to newly collected environmental data.

6.13 Features of the LSTM trained bot

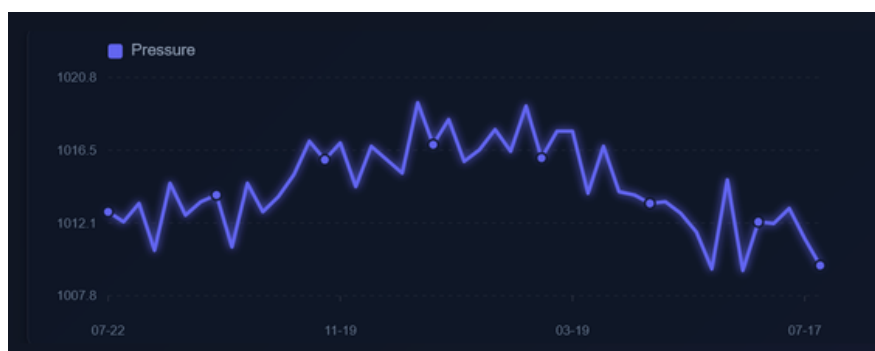
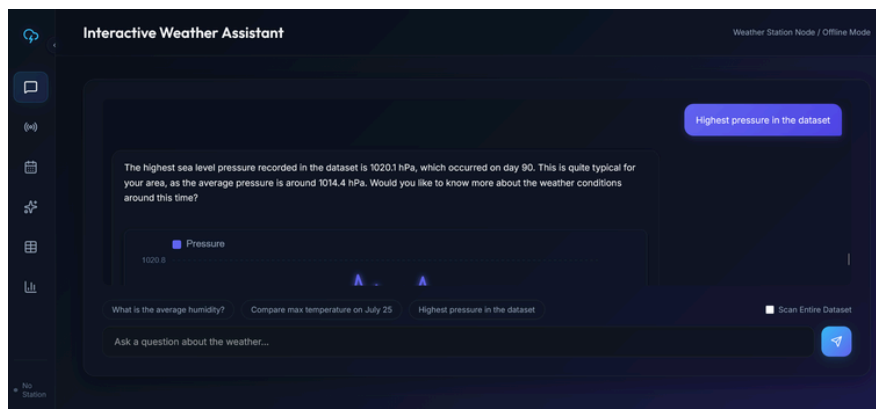
6.13.1 Historical Weather Information Retrieval

WeatherGPT enables users to retrieve historical weather information using natural language queries. The chatbot accesses a locally stored weather dataset and provides observations for parameters such as temperature, humidity, atmospheric pressure, precipitation, wind speed, wind direction, cloud cover, visibility, and other environmental variables. Users can request weather information for specific dates or custom date ranges without requiring knowledge of database queries or file structures.



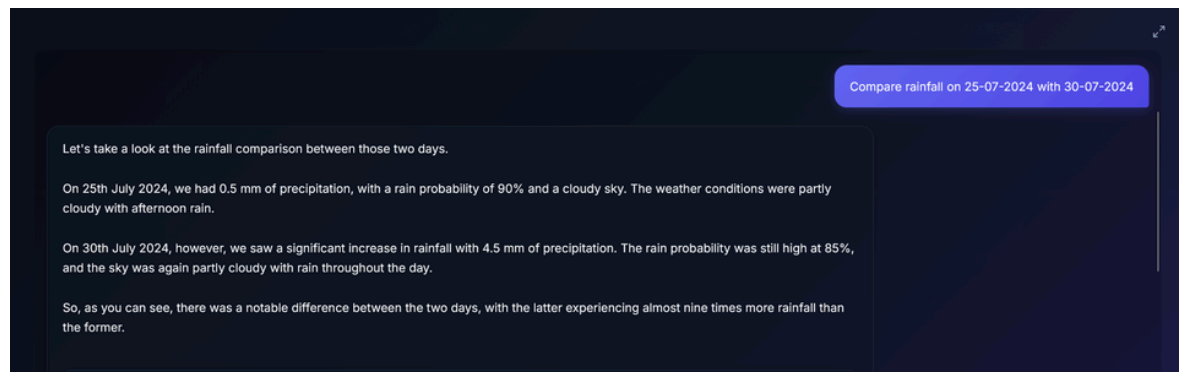
6.13.2 Statistical Weather Analysis

The chatbot supports statistical analysis of historical weather data by computing descriptive measures, including average, minimum, and maximum values for selected weather parameters. Statistical summaries can be generated for individual days, months, or user-defined periods, enabling efficient analysis of weather trends and seasonal variations through simple conversational interactions.



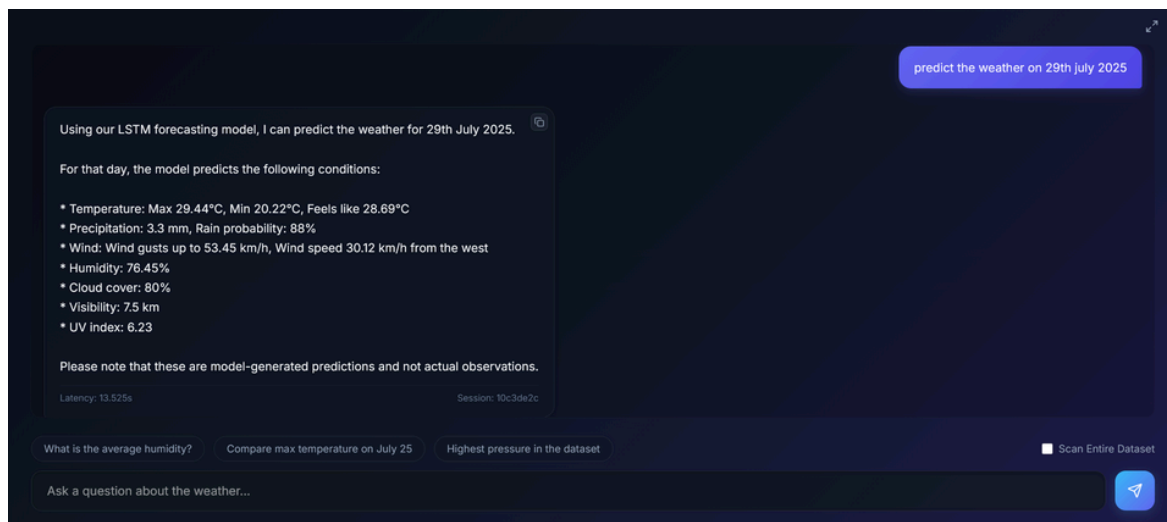
6.13.3 Multi-Date Weather Comparison

WeatherGPT provides comparative analysis of weather conditions across multiple dates. Users can compare environmental parameters recorded on different days to identify variations in atmospheric conditions, making the system useful for weather trend analysis and environmental studies.



6.13.4 LSTM-Based Weather Forecasting

To provide predictive weather intelligence, the chatbot incorporates a stacked Long Short-Term Memory (LSTM) neural network developed using the PyTorch framework. The forecasting model utilizes historical weather observations to predict future weather conditions for up to 30 days. Forecast results are presented through natural language responses, allowing users to obtain short-term weather predictions without directly interacting with the underlying machine learning model.

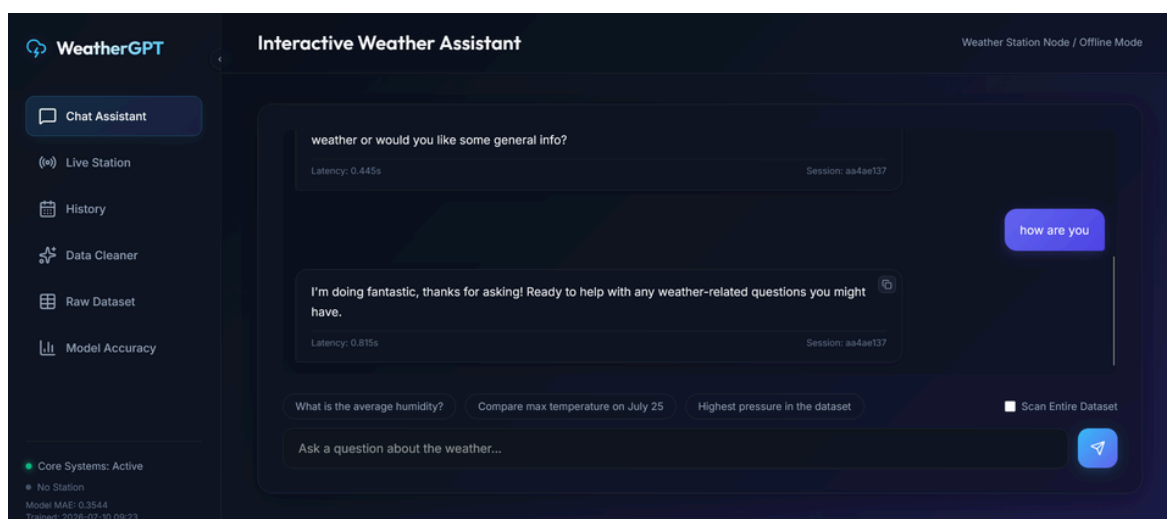


6.13.5 Natural Language Conversational Interface

The chatbot employs a Natural Language Processing (NLP) pipeline consisting of query parsing, intent classification, entity extraction, and decision-making modules. This enables users to communicate using everyday language rather than predefined commands, providing a more intuitive and user-friendly method for accessing weather information.

6.13.6 Interactive Web-Based Dashboard

In addition to the command-line interface, WeatherGPT provides a web-based dashboard for visualizing weather information. The dashboard presents historical observations, forecast results, interactive charts, and dataset health statistics, enabling users to explore environmental data through a graphical interface.



6.13.7 Offline Operation

The chatbot is designed to operate in offline as well using locally stored datasets and locally hosted software components. This capability makes the system suitable for deployment on edge computing platforms such as the Raspberry Pi, where reliable internet connectivity may not always be available.

6.13.8 Automatic Fallback Response Generation

To improve system reliability, WeatherGPT incorporates an automatic fallback mechanism that activates whenever the locally hosted Large Language Model (LLM) is unavailable. In such situations, the chatbot generates structured rule-based responses, ensuring uninterrupted operation and consistent user interaction without compromising the availability of weather services.

6.13.9 Modular and Scalable Architecture

WeatherGPT follows a modular software architecture in which the NLP pipeline, decision engine, forecasting module, response generation module, web interface, and MCP server are implemented as independent functional components. This modular design simplifies maintenance, facilitates future enhancements, and enables seamless integration of additional weather services, machine learning models, or AI applications with minimal modifications to the existing framework.

6.14 Design of new chatbot

The current implementation of WeatherGPT employs a stacked Long Short-Term Memory (LSTM) neural network for short-term weather forecasting. While the developed model demonstrates satisfactory performance in predicting weather conditions from historical temporal observations, it primarily captures sequential dependencies and treats each weather parameter independently. Consequently, the existing approach is not well suited for simultaneously modeling the complex relationships among multiple sensing nodes and heterogeneous environmental parameters present in an AIoT-based multi-modal weather monitoring station.

To address these limitations, an investigation was conducted to identify more advanced machine learning architectures capable of effectively handling multi-node and multi-sensor weather data. Based on this study, Graph Neural Networks (GNNs) were identified as a promising alternative due to their ability to represent complex spatial and relational dependencies using graph-based data structures. In a GNN framework, each sensing node can be represented as a graph node, while the interactions between sensors, environmental parameters, or geographically distributed weather stations can be modeled as graph edges. This enables the learning algorithm to capture both temporal patterns and spatial correlations that are difficult to represent using conventional LSTM models.

Currently, research and development efforts are focused on integrating a GNN-based forecasting model into the WeatherGPT framework. The objective is to develop a scalable prediction engine capable of simultaneously processing observations from multiple sensors and sensing nodes while improving forecasting accuracy and generalization. The proposed GNN-based approach is expected to enhance the capability of the AIoT weather monitoring system by enabling intelligent spatio-temporal weather prediction, efficient multi-node environmental analysis, and improved support for large-scale distributed weather monitoring applications.

